DATA STRUCTURES

UNIT 4: TREE

EMPLOYEE ID MANAGEMENT USING BST

Employee ID Insertion, Search and Sorted Display

ORIGINAL DOCUMENTATION

Subject	Data Structures
Unit	Unit 4 – Tree
Project	Employee ID Management Using BST
Version	Original
Programming Language	C++

Prepared for Second-Year Engineering – Data Structures Project Documentation

1. Introduction

A Binary Search Tree (BST) organizes data by key. For every node, IDs smaller than the node's ID are stored in the left subtree and larger IDs are stored in the right subtree. This project uses BST to insert employee IDs, search for a particular ID, and display IDs in sorted order.

2. Problem Statement

Develop a C++ program to manage employee records using a Binary Search Tree. The program should allow insertion of employee IDs, searching for an ID, and displaying IDs in ascending order.

3. Objectives

- To understand the structure and properties of a Binary Search Tree.
- To insert employee records according to BST ordering rules.
- To search for a required employee ID.
- To display employee IDs in ascending order using inorder traversal.

4. Data Structure Used

Binary Search Tree (BST): Each node has a key and links to its left and right child. For each node, values in the left subtree are smaller and values in the right subtree are larger. Inorder traversal (Left, Root, Right) displays keys in ascending order.

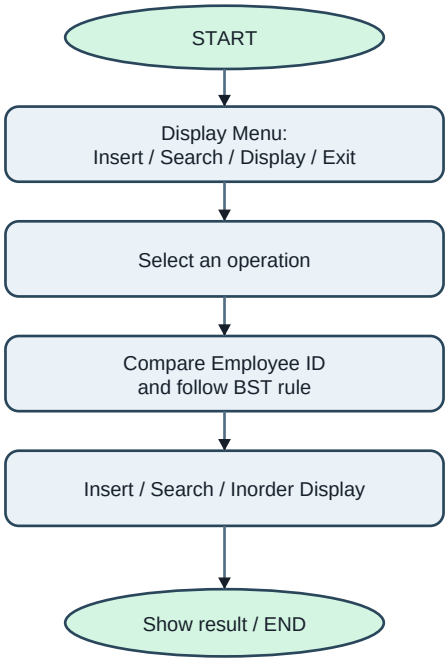
Node field	Purpose
Employee ID	Key used to arrange and search records.
Left pointer	Points to the left child containing smaller IDs.
Right pointer	Points to the right child containing larger IDs.

5. Algorithm

1. Start the program and initialize the BST root as NULL.
2. Display the menu: Insert, Search, Display Inorder, and Exit.
3. For insertion, read the employee ID.
4. If the tree is empty, create the root node; otherwise compare the ID with the current node.
5. If the ID is smaller, move to the left child; if larger, move to the right child.
6. Insert the new ID at the first available NULL position following the BST rule.
7. For search, compare the required ID with each visited node and move left or right accordingly.
8. For display, perform inorder traversal: Left subtree, Root, Right subtree.
9. Show the result and return to the menu until Exit is selected.
10. Stop the program.

6. Flowchart

The flowchart summarizes the main operations of the BST-based employee management system.



7. Working of the Project

The user enters an employee ID. The program compares it with the root and follows the left link when the ID is smaller or the right link when it is larger. Searching follows the same rule until the ID is found or a NULL link is reached. Inorder traversal visits the left subtree, node, and right subtree, displaying IDs in ascending order.

8. Programming Language and Concepts

- Programming language: C++.
- Structures and pointers to create BST nodes.
- Dynamic memory allocation for new nodes.
- Recursion or iterative traversal for insertion, search, and inorder display.
- Conditional statements and loops to implement menu-driven operations.

9. Advantages

- Efficient searching and insertion when the BST is reasonably balanced.
- Inorder traversal displays employee IDs in sorted order.
- Dynamic memory allocation allows the tree to grow as records are added.

10. Applications

- Employee record lookup and basic staff directories.
- Student ID or roll-number management.
- Understanding hierarchical data structures and ordered searching.

11. Sample Input and Output

Operation	Sample result
Insert	101, 105, 98, 110
Search	Search ID: 105 → Employee ID found
Inorder display	98, 101, 105, 110

12. Conclusion

The project demonstrates how a Binary Search Tree can organize employee IDs and support insertion, searching, and sorted display. It improves understanding of tree traversal, pointers, and dynamic data structures.

Time complexity: Average insertion/search is O(log n) in a reasonably balanced BST; worst case is O(n) for a skewed tree.